

Node Modules & Express.js

Node Modules

What is a Node Module

- Modules are simply libraries for Node.js
- Do not confuse this with a package which is one or more modules grouped together
 - Aka the Node Package Manager or npm

Things to Remember...

- Your code will use 'require' to bring in modules
 - We've seen this before with fs and other things
- Modules use 'exports' to make things available
 - Typically is made available throughout the runtime

Let's Make a Simple Module

- Create a file called “hello.js”
- Inside of that file write the statement:

```
module.exports = function() {  
  console.log("hello");  
}
```

- Module is a variable that represents the current module and exports is an object that will expose the entire module

Let's Make a Simple Module (cont.)

- Now create a file in the same directory called “app.js”
- Inside of app.js state the following

```
var app = require('./hello.js');  
  
app();
```

- Note: we cannot just state “hello.js” since Node needs the developer to specify the current directory

Creating an Async Function Using Modules

- Just like we learned in the previous modules, asynchronous functions will take in a function (callback) that one or two parameters
- We can define our own asynchronous functions as a module

Writing Async forEach

- Create a file called `forEachModule.js`
- Use `module.export` to create a function that has two parameters:
 - `arr`: an array to be iterated through
 - `callback`: this function will be called when the function completes
- Note: the parameters can be named whatever you want! They are just normal parameters in any JavaScript function

```
module.exports = function(arr, callback) {  
  
}
```


Now We'll Loop Through the Array...

- We will loop through the array that we passed in

```
for (var i = 0; i < arr.length; i++) {  
    // Do something here  
}
```

- Think about the functionality of the forEach loop
- It returns each element in the given array (and the element's index if you get the other parameter)

Add the Callback Function

```
module.exports = function(arr, callback) {  
  for (var i = 0; i < arr.length; i++) {  
    callback(arr[i], i);  
  }  
}
```

Express.js

What is Express.js

- Is a flexible Node.js application framework that provides a robust set of features for web and mobile applications
- It is open-source and maintained by the Node foundation
- Express does have a few competitors:
 - Rails: Used with Ruby
 - Django: Used with Python
- Express is much more flexible and and pluggable
- This is the 'E' in the ME(A||R||V)N stack

Express Tutorial

- Need to “npm install express” just like any other package
- Also must require it at the top of a file by using “var express = require('express');”
- Will typically then need to do “var app = express()”
- App will now be a huge JSON blob with a bunch of cool functions that will help with server-side programming
 - HTTP stuff
 - etc.

```
app.get('/', function(req, res){  
  res.send("Hello world!");  
});  
  
console.log("Navigate to http://localhost:3000/")  
  
app.listen(3000);
```

.get() breakdown

- The .get() function takes in two parameters
 - 1st: is a path or “route” which directly correlates to the url in the browser
 - If just a “/” is provided, that means the “root” directory just like in any filesystem
 - 2nd: a callback function that has two parameters
 - Req: which represents the data for the HTTP request like parameters, body, head
 - Res: represents the response that the Express app sends when a HTTP request is received

1st Parameter: path

- In our given example we just use “/” as the root directory
- What if we replaced with “/testdir”
 - It would route to that given url or directory
- These are how “endpoints” are created in a RESTful API
 - Will discuss further after the midterm

res.send()

- The send function takes an object as a parameter and sends it to the request client
 - Remember, we are now the server, not the client so we will be providing information
 - This is how APIs are built!
- In our example, we are just serving the string

app.listen()

- This function will bind and listen for connections on that given port
- Can also pass it a function that runs when the app starts listening

Switching it up

- What happens if we change the path?
- What happens if we serve something other than a String?

Accessing Params in Express

- `/api/something?name=jal&job=professor`
- Can access easily:
 - `req.query.name`
 - `req.query.job`

Express Reference

<http://expressjs.com/en/api.html#app.get>

- Other Verbs
 - POST
 - PUT
 - DELETE

Simple Sessions

- Can leverage express to manage user sessions in your page
 - Express-session: Actually manages the current session
 - Cookie-parser: Middleware that parses cookies from a client (incoming request)

Express Session

- Define a session that express recognizes

```
var app = express();

app.use(cookieParser());

app.use(session({
  secret: "some_cookie", //hash representing the session
  name: "cookie_name",
  resave: true, //session to be saved back in session store
  saveUninitialized: true //forces uninitialized session to be saved in store
}));
```

Utilizing a Session

- Simply adding to the session variable
- Nifty to track new recent logins
 - Try out: tracking time!

```
app.get('/', function(req, res){
  if(req.session.page_views){
    req.session.page_views++;
    res.send("You visited this page " + req.session.page_views + " time(s)");
  } else {
    req.session.page_views = 1;
    res.send("Welcome to this page for the first time!");
  }
});
app.listen(3000);
```